

Contents

WHAT IS LLM ?.....	3
Transformers.....	5
11. Stacked Transformer Layers.....	11
12. Output Linear Layer.....	11
13. Softmax.....	12
Transformers Main Parts.....	13
Tokenization.....	14
4 Stages of Training LLMs from Scratch.....	22
Stage 0: Randomly Initialized Model.....	23
Stage 1: Pre-training.....	24
Stage 2: Instruction Fine-Tuning.....	24
Stage 3: Preference Fine-Tuning (PFT).....	25
Stage 4: Reasoning Fine-Tuning.....	26
Quick Recap.....	26
A Couple of Things Worth Adding.....	27
1. Conditional Probability — The Foundation.....	27
2. How This Applies to LLMs.....	27
3. What the Model Actually Learns.....	28
4. The Problem With Always Picking the Best Word.....	28
5. Temperature and Sampling.....	28
Interview-Ready Notes (Simple 2-Line Answers).....	28
One-Line Summary.....	29
Overview.....	30
7 LLM Generation Parameters.....	31
0. The Big Picture (say this first if asked to explain generation).....	35
1. Max Tokens.....	36
2. Temperature.....	36
3. Top-k.....	37
4. Top-p (Nucleus Sampling).....	37
5. Frequency Penalty.....	38
6. Presence Penalty.....	38

7. Stop Sequences.....	39
Quick Comparison Table (great for a whiteboard answer).....	39
Likely Interview Questions + Simple Answers.....	40
4 LLM Text Generation Strategies.....	41
Closing summary (with analogy).....	44
3 Techniques to Train An LLM Using Another LLM.....	44
Quick way to wrap it up.....	48
Transformer vs. Mixture of Experts (MoE) in LLMs.....	49
Core Idea.....	50
How Experts Get Selected — The Router.....	50
Challenge 1: Expert Imbalance (Rich-Get-Richer Problem).....	51
Challenge 2: Uneven Token Load.....	51
Key Outcome.....	51
Simple Explanation with Analogy (Interview-Ready).....	51
Analogy: A Hospital With Specialist Doctors.....	51
How I'd Explain It Out Loud in an Interview.....	52
Quick Closing Line for Interviews.....	52

WHAT IS LLM ?

What is an LLM?

Imagine someone begins a sentence:



"Once upon a..."

You naturally think "time."

Or they say:

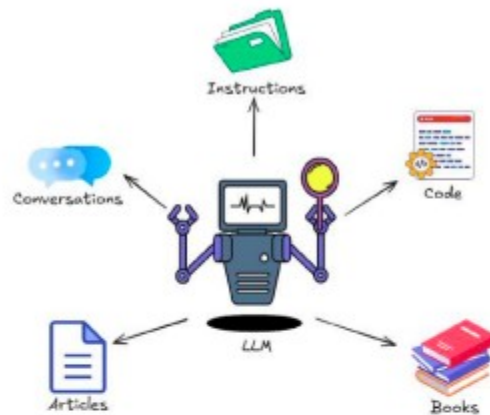
"The capital of France is..."

You immediately think "Paris."



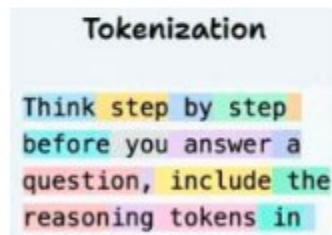
This simple act of predicting what comes next is the foundation of how large language models (LLMs) operate.

They learn to make these predictions by reading enormous amounts of text: books, articles, scientific papers, code, conversations, and instructions.



With enough exposure, the model becomes remarkably good at continuing any piece of text in a coherent, meaningful way.

At the technical level, an LLM processes text in small units called tokens. A token may be a word, part of a word or even punctuation.



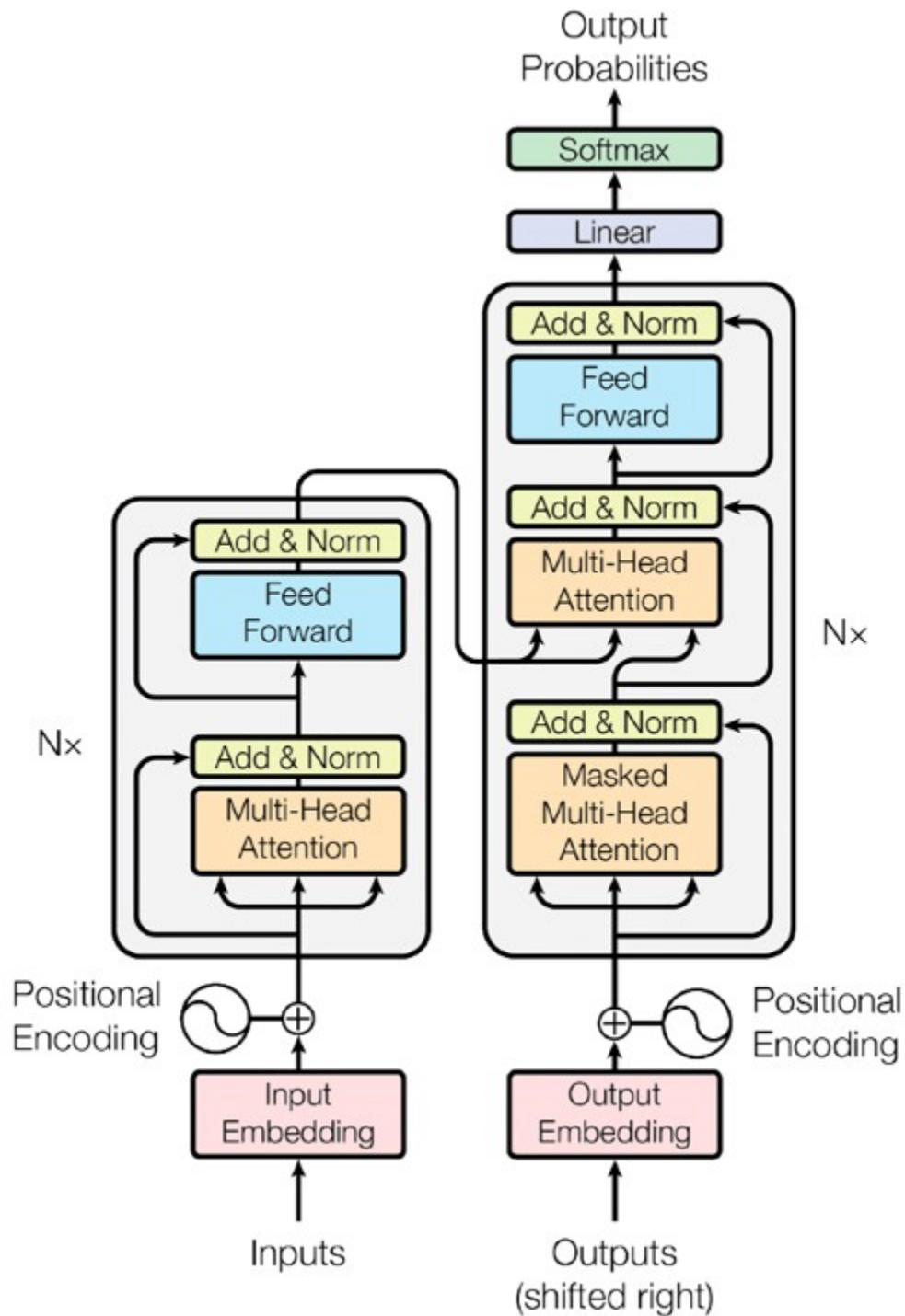
The model looks at the tokens so far and predicts the next one. Repeating this process generates full answers, explanations, or code.

Everything an LLM does from summarizing a document, generating a function or explaining a concept emerges from choosing the next token that best fits the patterns it has learned.

Transformers

To formalise, a large language model is a Transformer-based neural network trained on massive text corpora to predict the next token in a sequence and through this process acquires the ability to understand, generate and reason with human language.

For indepth notes refer finetuning notes word doc prepared by me



A **Transformer** is a deep learning architecture that processes sequences using **Attention Mechanism** instead of recurrence. It is the core architecture used in modern GenAI models.

Sentence:

“The cat sat on the mat”

We will see **what happens to this sentence inside the transformer**.

1. Tokenization

Purpose (one line):

Break the sentence into smaller pieces called **tokens**.

What happens

"The cat sat on the mat"

becomes

["The", "cat", "sat", "on", "the", "mat"]

Some models may convert them to subwords like

["The", "cat", "sat", "on", "the", "mat"]

Now each token will be processed individually.

2. Input Embedding

Purpose (one line):

Convert tokens into **numeric vectors** so the neural network can process them.

What happens

Each word becomes a vector.

Example (simplified):

The → [0.21, 0.54, 0.11, ...]

cat → [0.33, 0.19, 0.72, ...]

sat → [0.48, 0.67, 0.12, ...]

These vectors represent **semantic meaning** of words.

3. Positional Encoding

Purpose (one line):

Add information about **word order** because transformers process tokens in parallel.

What happens

The model adds position information:

The → embedding + position(1)

cat → embedding + position(2)

sat → embedding + position(3)

Now the model knows:

cat comes after The

sat comes after cat

Without this, the model would treat the sentence as a **bag of words**.

4. Linear Projection → Query, Key, Value

Purpose (one line):

Create **Query, Key, Value vectors** used to calculate attention.

Each token vector is converted into:

Query (Q)

Key (K)

Value (V)

Example:

cat → Q_cat, K_cat, V_cat

sat → Q_sat, K_sat, V_sat

These vectors help the model decide **which words should pay attention to which words**.

5. Scaled Dot Product Attention

Purpose (one line):

Compute **how strongly each word relates to every other word**.

Example:

When processing **sat**, the model checks which words are important.

Attention scores may look like:

Word Attention from "sat"

The	0.1
cat	0.6
sat	0.2
on	0.05
mat	0.05

Meaning:

"sat" strongly relates to "cat"

So the model understands:

cat is the one sitting

This is the core idea of **Self-Attention**.

6. Multi-Head Attention

Purpose (one line):

Allow the model to capture **multiple relationships simultaneously**.

Instead of one attention calculation, the model runs **many attention heads**.

Example heads might learn:

Head 1 → subject relationship

Head 2 → verb relationship

Head 3 → positional relation

Head 4 → semantic similarity

Example:

Head1: cat ↔ sat
Head2: sat ↔ mat
Head3: cat ↔ the

All heads are combined to create a richer representation.

7. Add & Residual Connection

Purpose (one line):

Add the original input back to prevent losing information.

Formula idea:

$\text{output} = \text{attention_output} + \text{original_input}$

This helps deep networks train better.

8. Layer Normalization

Purpose (one line):

Normalize values to stabilize training and speed convergence.

It keeps values within a controlled range so the network doesn't explode or vanish.

9. Feed Forward Neural Network (FFN)

Purpose (one line):

Apply nonlinear transformations to refine token representations.

Each token passes through a small neural network:

$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$

Example improvement:

"sat" representation now strongly connects with "cat"

The model better understands **sentence meaning**.

10. Add & LayerNorm Again

Purpose (one line):

Stabilize and preserve information after the feed-forward step.

Again:

$\text{output} = \text{FFN_output} + \text{previous_input}$

Then normalization is applied.

11. Stacked Transformer Layers

Purpose (one line):

Repeat the attention + FFN process multiple times to build deeper understanding.

Typical models stack:

6 layers
12 layers
24 layers
96 layers (in large LLMs)

Each layer captures **more complex relationships**.

Example progression:

Layer 1 → basic word relationships
Layer 4 → subject-verb structure
Layer 8 → sentence meaning
Layer 12 → deeper context

12. Output Linear Layer

Purpose (one line):

Convert the final hidden representation into **scores for each vocabulary word**.

Example predicted scores for next word:

mat → 0.52
floor → 0.22

chair → 0.10
table → 0.05

13. Softmax

Purpose (one line):

Convert scores into probabilities.

Example:

mat = 52%
floor = 22%
chair = 10%

The model predicts:

"mat"

Full Transformer Flow (Simple)

Sentence
↓
Tokenization
↓
Input Embedding
↓
Positional Encoding
↓
Q, K, V creation
↓
Self Attention
↓
Multi Head Attention
↓
Add & Norm
↓
Feed Forward Network
↓
Add & Norm
↓
Repeat layers
↓
Linear Layer
↓
Softmax
↓
Next token prediction

Component	Purpose
Tokenization	Split text into tokens
Embedding	Convert tokens to vectors
Positional Encoding	Preserve word order
Self Attention	Learn relationships between words
Query/Key/Value	Calculate attention relevance
Multi Head Attention	Learn multiple relationships simultaneously
Add & Norm	Stabilize training
Feed Forward	Extract deeper features
Transformer Layers	Build hierarchical understanding
Softmax Output	Generate prediction

One Sentence Summary

A **Transformer** processes text by converting tokens into vectors and repeatedly applying **Self-Attention** and feed-forward layers so each word understands its relationship with every other word.

Transformers Main Parts

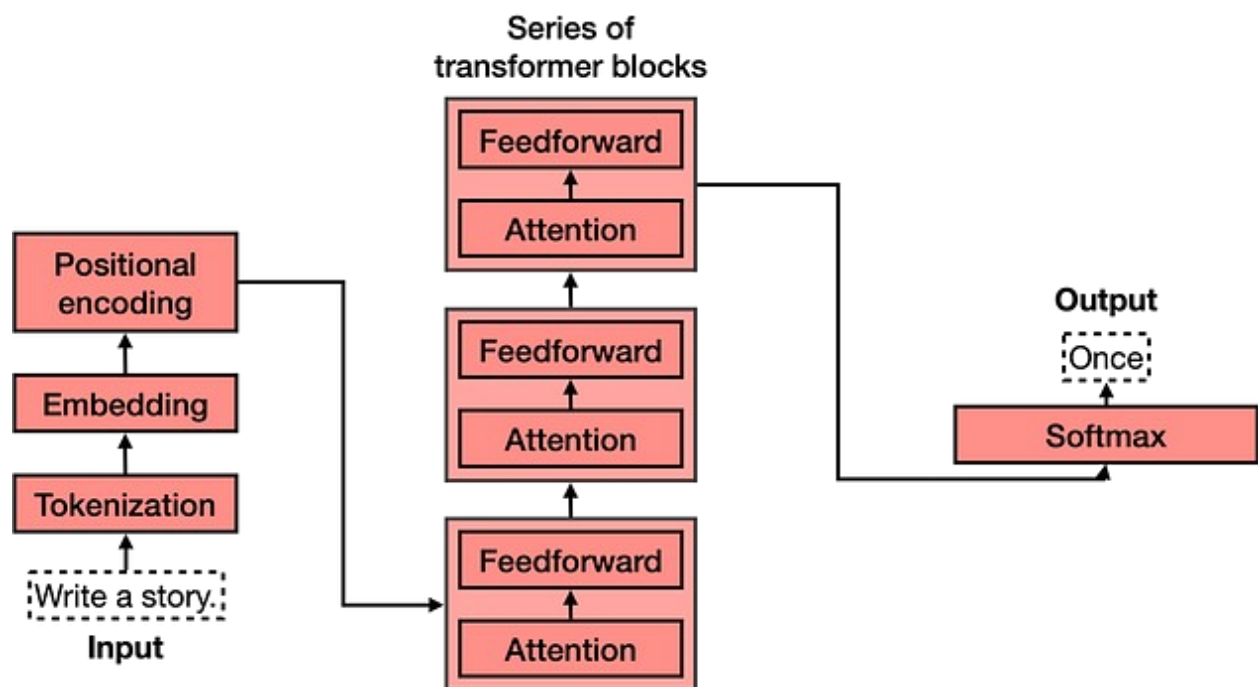
<https://medium.com/@amanatulla1606/transformer-architecture-explained-2c49e2257b4c>

Now that we know what transformers do, let's get to their architecture. If you've seen the architecture of a transformer model, you may have jumped in awe like I did the first time I saw it, it looks quite complicated! However, when you break it down into its most important parts, it's not so bad. The transformer has 4 main parts:

- Tokenization
- Embedding

- Positional encoding
- Transformer block (several of these)
- Softmax

The fourth one, the transformer block, is the most complex of all. Many of these can be concatenated, and each one contains two main parts: The attention and the feedforward components.



Tokenization

Tokenization is the most basic step. It consists of a large dataset of tokens, including all the words, punctuation signs, etc. The tokenization step takes every word, prefix, suffix, and punctuation signs, and sends them to a known token from the library.

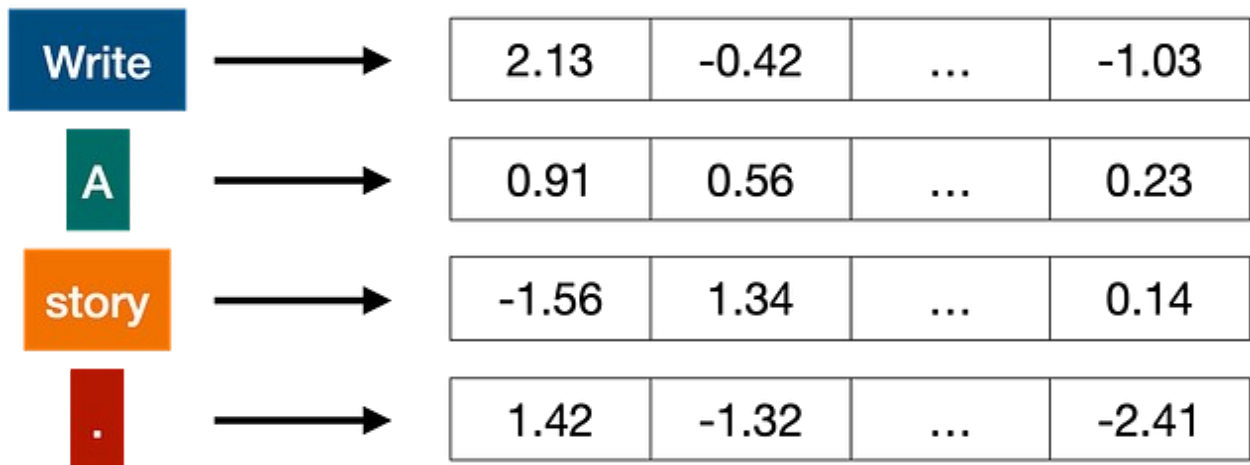
Tokenization



Embedding

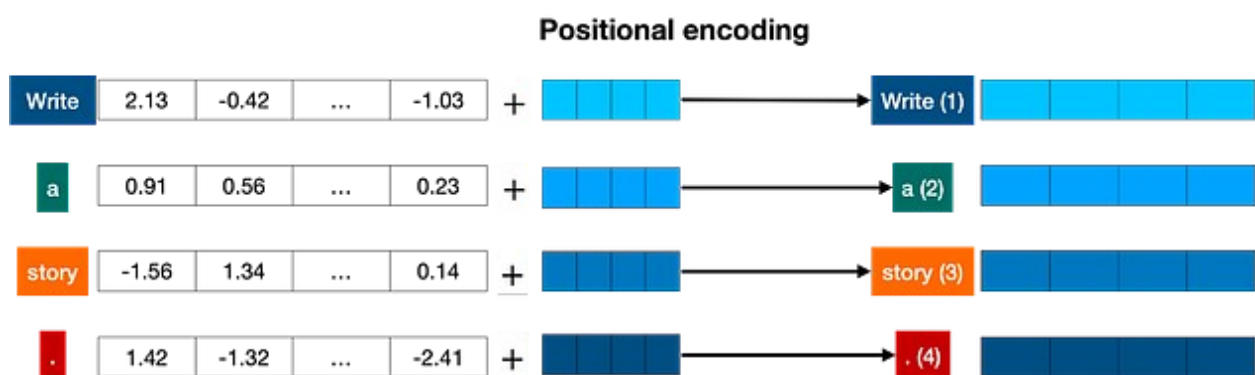
Once the input has been tokenized, it's time to turn words into numbers. For this, we use an embedding. In a previous chapter you learned about how text embeddings send every piece of text to a vector (a list) of numbers. If two pieces of text are similar, then the numbers in their corresponding vectors are similar to each other (componentwise, meaning each pair of numbers in the same position are similar). Otherwise, if two pieces of text are different, then the numbers in their corresponding vectors are different.

Embedding



Positional encoding

Once we have the vectors corresponding to each of the tokens in the sentence, the next step is to turn all these into one vector to process. The most common way to turn a bunch of vectors into one vector is to add them, componentwise. That means, we add each coordinate separately. For example, if the vectors (of length 2) are $[1,2]$, and $[3,4]$, their corresponding sum is $[1+3, 2+4]$, which equals $[4, 6]$. This can work, but there's a small caveat. Addition is commutative, meaning that if you add the same numbers in a different order, you get the same result. In that case, the sentence "I'm not sad, I'm happy" and the sentence "I'm not happy, I'm sad", will result in the same vector, given that they have the same words, except in different order. This is not good. Therefore, we must come up with some method that will give us a different vector for the two sentences. Several methods work, and we'll go with one of them: positional encoding. **Positional encoding consists of adding a sequence of predefined vectors to the embedding vectors of the words.** This ensures we get a unique vector for every sentence, and sentences with the same words in different order will be assigned different vectors. In the example below, the vectors corresponding to the words "Write", "a", "story", and "." become the modified vectors that carry information about their position, labeled "Write (1)", "a (2)", "story (3)", and ". (4)".



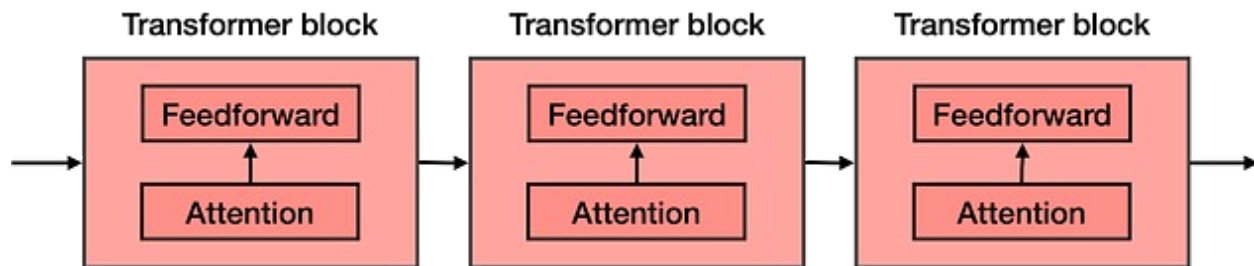
Transformer block

Let's recap what we have so far. The words come in and get turned into tokens (tokenization), tokenized words are turned into numbers (embeddings), then order gets taken into account (positional encoding). This gives us a vector for every token that we input to the model. Now, the next step is to predict the next word in this sentence. This is done with a really really large neural network, which is trained precisely with that goal, to predict the next word in a sentence.

We can train such a large network, but we can vastly improve it by adding a key step: the attention component. Introduced in the seminal paper *Attention is All you Need*, it is one of the key ingredients in transformer models, and one of the reasons they work so well. Attention is explained in the previous section, but for now, imagine it as a way to add context to each word in the text.

The attention component is added at every block of the feedforward network. Therefore, if you imagine a large feedforward neural network whose goal is to predict the next word, formed by several blocks of smaller neural networks, an attention component is added to each one of these blocks. Each component of the transformer, called a transformer block, is then formed by two main components:

- The attention component.
- The feedforward component.



Attention

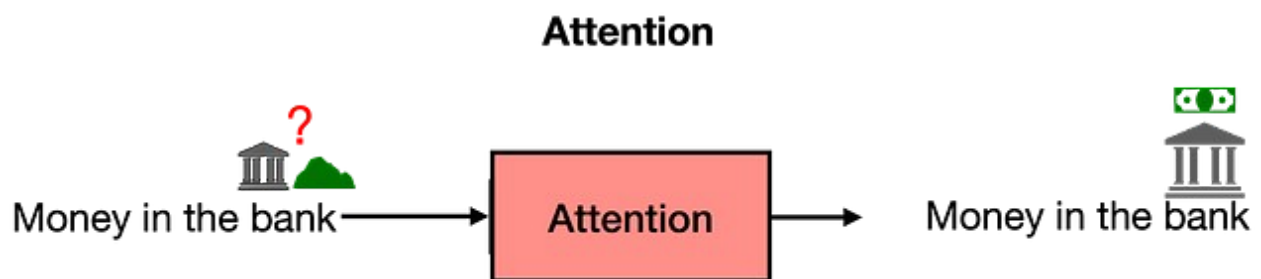
The next step is attention. As you learned attention mechanism deals with a very important problem: the problem of context. Sometimes, as you know, the same word can be used with different meanings. This tends to confuse language models, since an embedding simply sends words to vectors, without knowing which definition of the word they're using.

Attention is a very useful technique that helps language models understand the context. In order to understand how attention works, consider the following two sentences:

- Sentence 1: The bank of the river.
- Sentence 2: Money in the bank.

As you can see, the word 'bank' appears in both, but with different definitions. In sentence 1, we are referring to the land at the side of the river, and in the second one to the institution that holds money. The computer has no idea of this, so we need to somehow inject that knowledge into it. What can help us? Well, it seems that the other words in the sentence can come to our rescue. For the first sentence, the words 'the', and 'of' do us no good.

But the word 'river' is the one that is letting us know that we're talking about the land at the side of the river. Similarly, in sentence 2, the word 'money' is the one that is helping us understand that the word 'bank' is now referring to the institution that holds money.



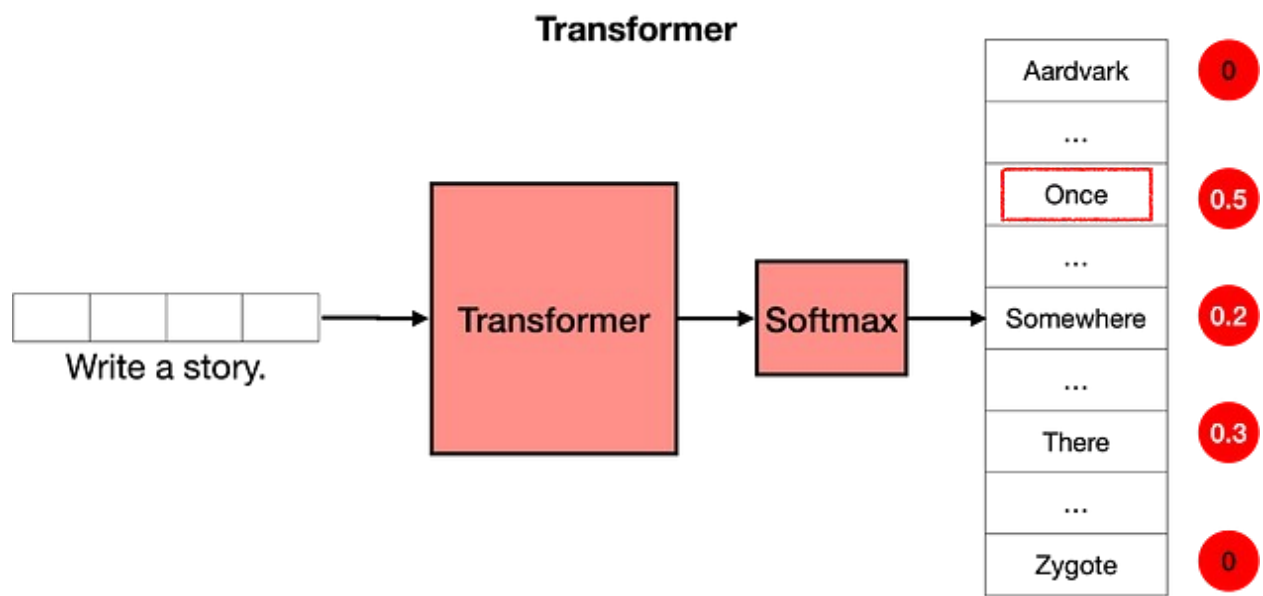
In short, what attention does is it moves the words in a sentence (or piece of text) closer in the word embedding. In that way, the word "bank" in the sentence "Money in the bank" will be moved closer to the word "money". Equivalently, in the sentence "The bank of the river", the word "bank" will be moved closer to the word "river". That way, the modified word "bank" in each of the two sentences will carry some of the information of the neighboring words, adding context to it.

The attention step used in transformer models is actually much more powerful, and it's called **multi-head attention**. In multi-head attention, several different embeddings are used to modify the vectors and add context to them. Multi-head attention has helped language models reach much higher levels of efficacy when processing and generating text.

The Softmax Layer

Now that you know that a transformer is formed by many layers of transformer blocks, each containing an attention and a feedforward layer, you can think of it as a large neural network that predicts the next word in a sentence. The transformer outputs scores for all the words, where the highest scores are given to the words that are most likely to be next in the sentence.

The last step of a transformer is a softmax layer, which turns these scores into probabilities (that add to 1), where the highest scores correspond to the highest probabilities. Then, we can sample out of these probabilities for the next word. In the example below, the transformer gives the highest probability of 0.5 to “Once”, and probabilities of 0.3 and 0.2 to “Somewhere” and “There”. Once we sample, the word “once” is selected, and that’s the output of the transformer.



The softmax layer turns the scores into probabilities, and these are used to pick the next word in the text.

Now what? Well, we repeat the step. We now input the text “Write a story. Once” into the model, and most likely, the output will be “upon”. Repeating this step again and again, the transformer will end up writing a story, such as “Once upon a time, there was a ...”.

Post Training

Now that you know how transformers work, we still have a bit of work to do. Imagine the following: You ask the transformer “What is the capital of Algeria?”. We would love for it to answer “Algiers”, and move on. However, the transformer is trained on the entire internet. The internet is a big place, and it’s not necessarily the best question/answer repository. Many pages, for example, would have long lists of questions without answers. In this case, the next sentence after “What is the capital of Algeria?” could be another question, such as “What is the population of Algeria?”, or “What is the capital of Burkina Faso?”. The transformer is not a human who thinks about their responses, it simply mimics what it sees on the internet (or any dataset that has been provided). So how do we get the transformer to answer questions?

The answer is post-training. In the same way that you would teach a person to do certain tasks, you can get a transformer to perform tasks. Once a transformer is trained on the entire internet, then it is trained again on a large dataset which corresponds to lots of questions and their respective answers. Transformers (like humans), have a bias towards the last things

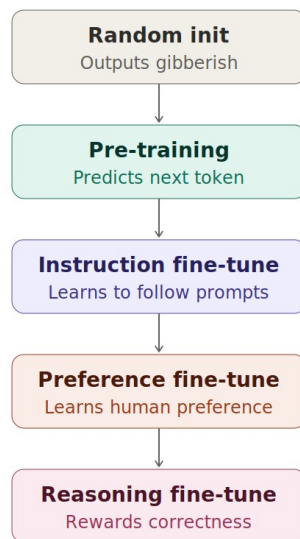
they've learned, so post-training has proven a very useful step to help transformers succeed at the tasks they are asked to.

Post-training also helps with many other tasks. For example, one can post-train a transformer with large datasets of conversations, in order to help it perform well as a chatbot, or to help us write stories, poems, or even code.

4 Stages of Training LLMs from Scratch

A simplified explanation, based on "4 Stages of Training LLMs from Scratch" by Avi Chawla (Daily Dose of Data Science)

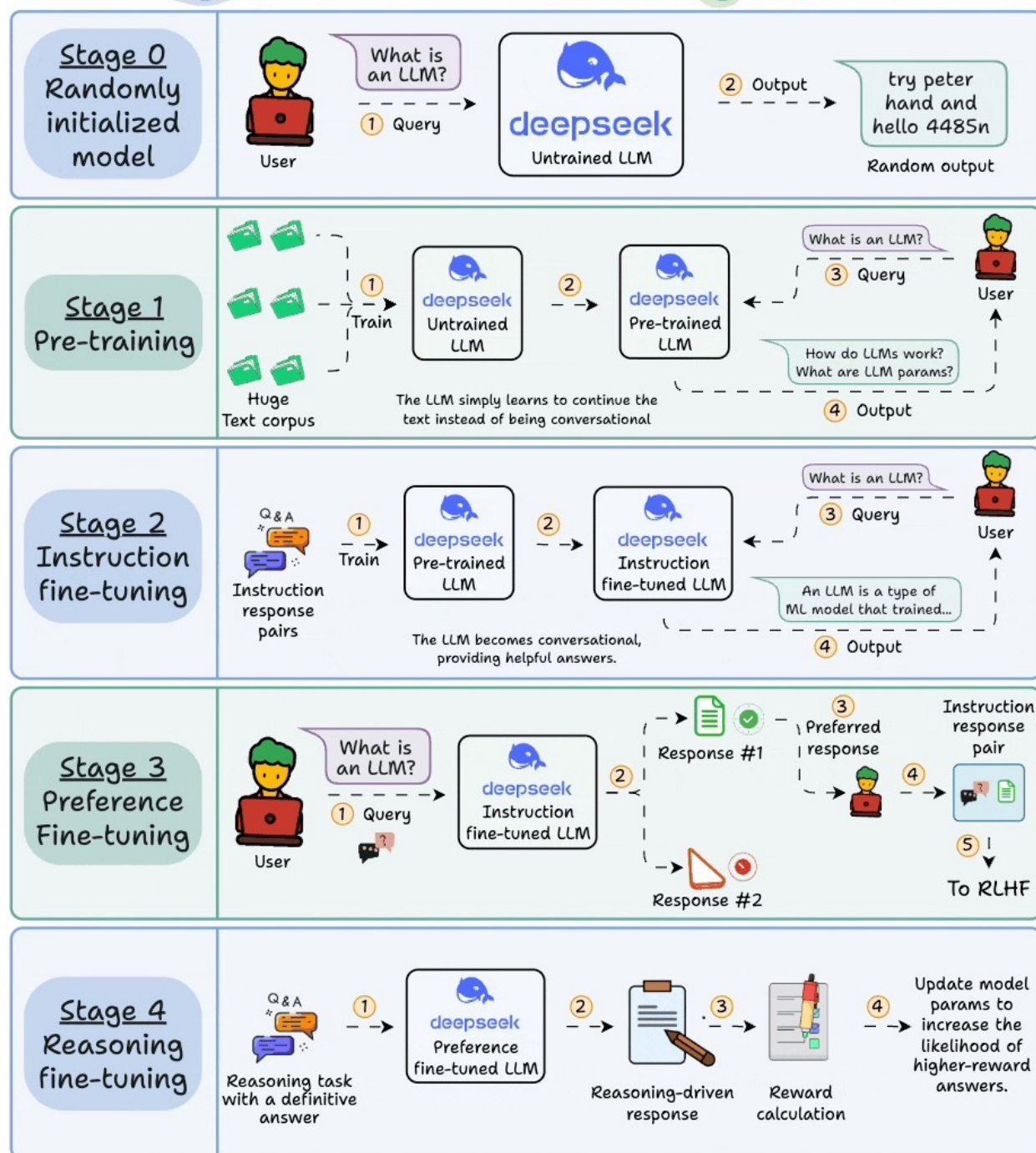
<https://blog.dailydoseofds.com/p/4-stages-of-training-llms-from-scratch>



Each stage builds on the model produced by the one before it

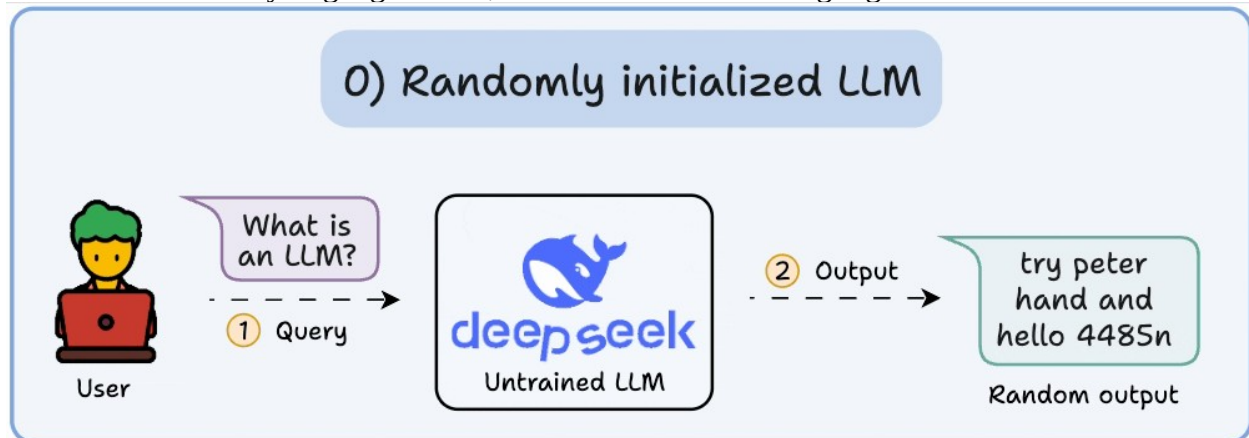
Building a large language model isn't just one training run. It happens in stages, and each stage builds on the one before it. Here are the 4 main stages, in simple words

4 stages of LLM Training



Stage 0: Randomly Initialized Model

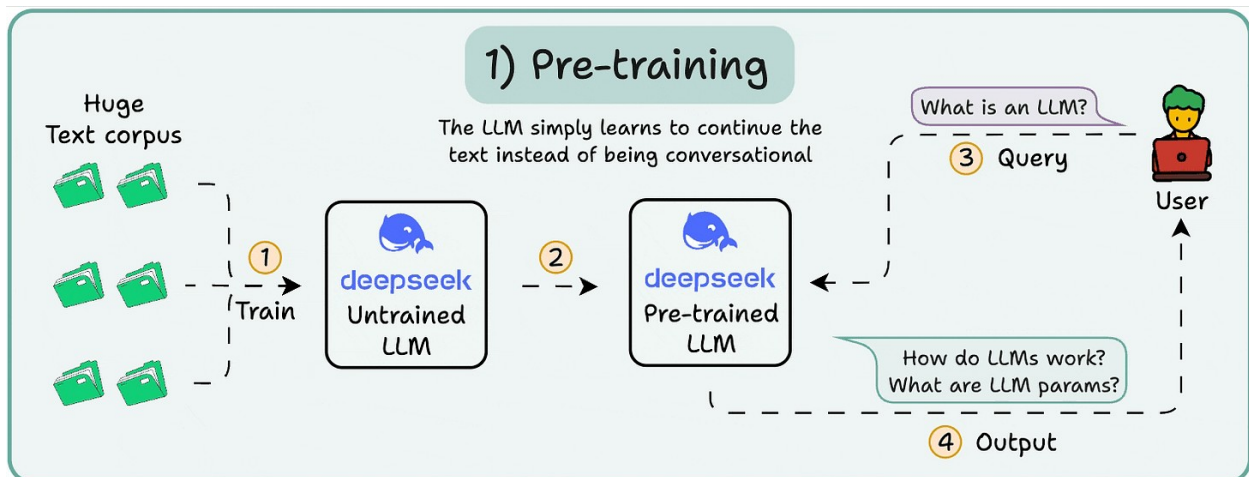
Before any training, the model knows nothing. Its weights are just random numbers. If you ask it "What is an LLM?" you get gibberish, like random words strung together.



Stage 1: Pre-training

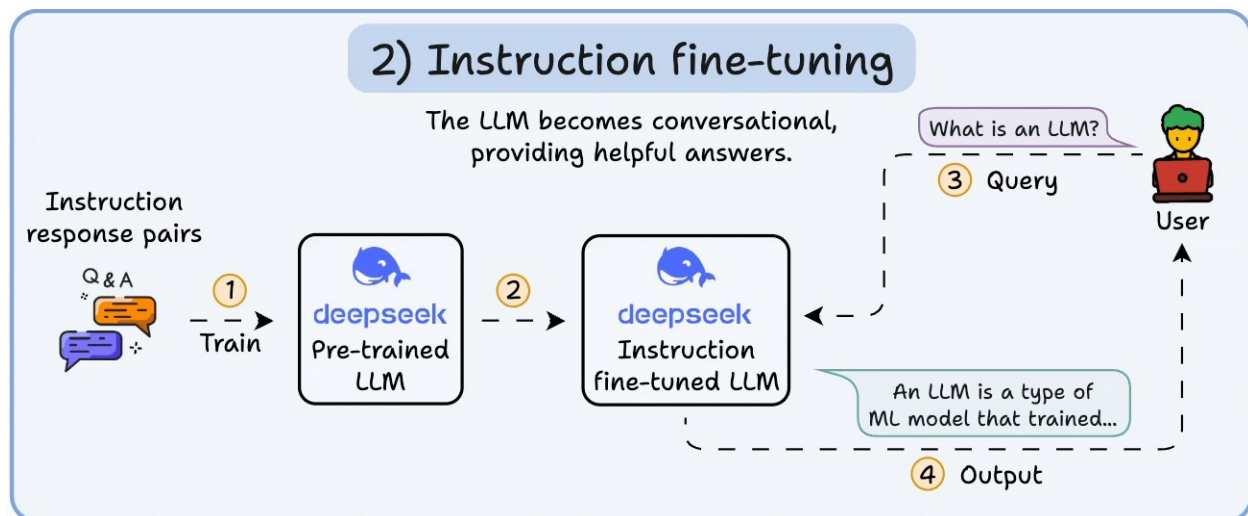
The model is trained on huge amounts of text from the internet. Its only job is to guess the next word, over and over. Doing this billions of times teaches it grammar, facts, and how language works.

Problem: it's not good at conversation yet. Ask it a question, and it may just continue the sentence instead of answering — because that's all it learned to do: continue text.



Stage 2: Instruction Fine-Tuning

Now the model is trained again, this time on examples of instructions paired with good responses (question and answer pairs). This teaches it to actually follow what you ask and reply properly, instead of just continuing text.



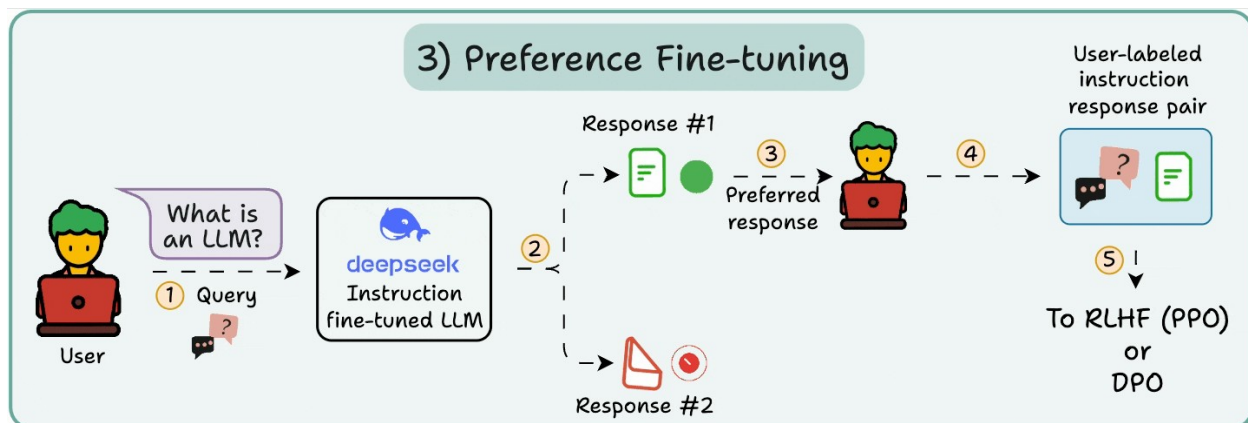
After this stage, the model can:

- Answer questions
- Summarize text
- Write code

At this point, the model has learned from pretty much all the internet text and human-written instruction examples available. To improve further, the next stages use reinforcement learning (RL) instead of just more examples.

Stage 3: Preference Fine-Tuning (PFT)

You may have seen an AI chat app show two answers and ask "which one do you prefer?" That click isn't just feedback — it's valuable training data about human preference.



Here's how it works:

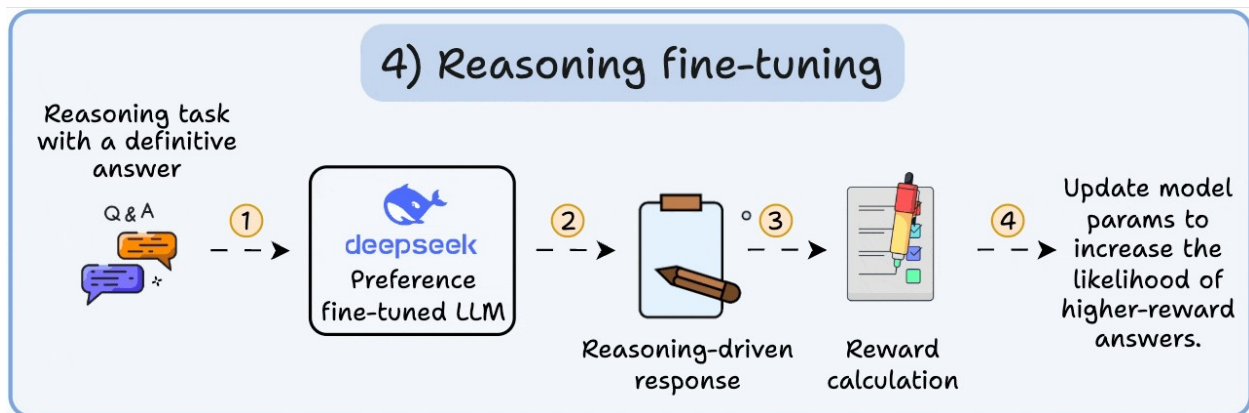
- The model gives two different answers to the same question.
- A person picks the better one.
- A separate "reward model" learns to predict which answers humans prefer.

- The main model is then updated using reinforcement learning to produce more answers like the ones humans preferred.

This whole process is called RLHF — Reinforcement Learning with Human Feedback. It helps the model give better answers even when there's no single "correct" answer, like tone, helpfulness, or safety. (The classic algorithm used here is called PPO. Many newer models now use a simpler alternative called DPO.)

Stage 4: Reasoning Fine-Tuning

For tasks like math or logic, there's usually one clear correct answer and clear steps to get there. So instead of needing human preference, we can just check: was the answer right or wrong?



Steps:

- The model generates an answer.
- The answer is compared with the known correct answer.
- If it's correct, the model gets a reward, and it's trained to do more of that.

This approach is called RLVR — Reinforcement Learning with Verifiable Rewards, since the reward is based on checking correctness rather than human opinion. A well-known technique here is GRPO, used by DeepSeek.

Those were the 4 stages of training an LLM.

- Start with a randomly initialized model.
- Pre-train it on large-scale corpora.
- Use instruction fine-tuning to make it follow commands.
- Use preference & reasoning fine-tuning to sharpen responses.

Quick Recap

- Start with a randomly initialized model that knows nothing.
- Pre-train it on huge text data so it learns language and facts.
- Instruction fine-tune it so it follows commands and answers properly.
- Preference fine-tune it (RLHF) so its answers match what humans actually prefer.
- Reasoning fine-tune it (RLVR) so it gets better at tasks with a clear right answer, like math.

A Couple of Things Worth Adding

The original blog covers the core 4 stages accurately — this is the standard framework used across the industry. A couple of small additions for a fuller picture:

- PPO isn't the only algorithm for preference fine-tuning anymore — DPO (Direct Preference Optimization) is a simpler, increasingly common alternative.
- In practice, companies often loop back between stages (e.g. more preference tuning after reasoning tuning) rather than doing each stage just once.

How Do LLMs Work?

A simplified explanation, based on "How Do LLMs Work?" by Avi Chawla (Daily Dose of Data Science)

1. Conditional Probability — The Foundation

Conditional probability just means: the chance of one thing happening, given that something else already happened.

Example: If it's cloudy (event B), it's more likely to rain (event A). We write this as $P(A|B)$ — "probability of A given B." Clouds don't guarantee rain, but they raise the chance.

2. How This Applies to LLMs

An LLM's core job is to predict the next word in a sentence. This is exactly a conditional probability problem: given the words so far (the context), what's the most likely next word?

For every possible next word, the model calculates a probability — how likely that word is to come next, given the context. The word (or words) with high probability are strong candidates for what comes next.

3. What the Model Actually Learns

During training, the model learns a giant probability distribution over sequences of words — basically, patterns of which words tend to follow which other words, across huge amounts of text.

The model's trained weights are literally the parameters of this probability distribution. Training this (called pre-training) is done in a supervised way — the model repeatedly guesses the next word and gets corrected based on the real next word in the training text.

4. The Problem With Always Picking the Best Word

If the model always picks the single most probable next word every time (called greedy decoding), the output becomes repetitive and boring — and it kills the model's creativity.

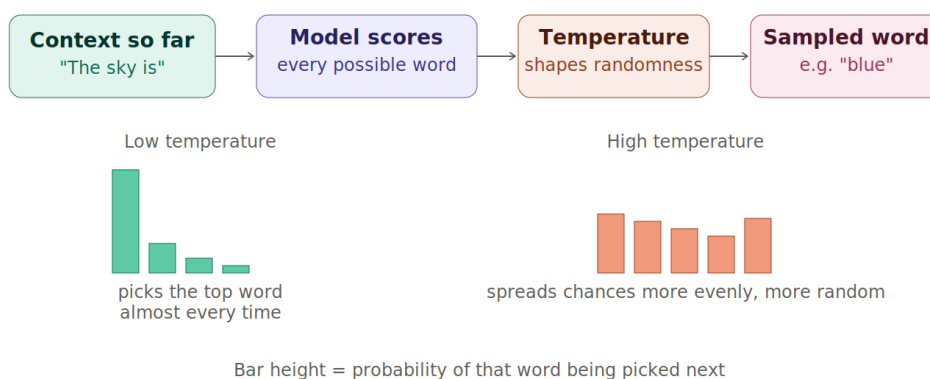
This is the problem that temperature and sampling are designed to fix.

5. Temperature and Sampling

Instead of always choosing the top word, the model samples — it randomly picks a word based on the probabilities, so less likely words still get a chance sometimes.

Temperature is a setting that controls how random this sampling is:

- Low temperature → probabilities stay concentrated on the top word(s). Output is close to always picking the best word (safe, focused, less creative).
- High temperature → probabilities spread out more evenly across many words. Output becomes more random and creative, but can be less coherent.



Interview-Ready Notes (Simple 2-Line Answers)

Conditional Probability:

The chance of one event happening given that another related event already happened. Written as $P(A|B)$ — probability of A given B.

Next-Word Prediction in LLMs:

The model treats next-word prediction as a conditional probability problem — given the context so far, it scores how likely each possible next word is.

What the Model Learns:

The model learns a huge probability distribution over word sequences during training. Its trained weights are the parameters of this distribution.

Greedy Decoding Problem:

Always picking the highest-probability word leads to repetitive, robotic output with no creativity.

Temperature:

A setting that controls how random the next-word sampling is — low temperature is focused and predictable, high temperature is more random and creative.

One-Line Summary

An LLM predicts the next word using conditional probability, and temperature controls how randomly it samples from those probabilities — low temperature plays it safe, high temperature gets more creative.

7 LLM Generation Parameters

Every response an LLM produces is shaped by a handful of parameters set at generation time. This guide walks through the 7 parameters that matter most, with an original diagram illustrating each one.

Overview

Knowing how to tune generation parameters is what separates a sharp, controlled LLM output from a haphazard one. Below is a quick-reference table, followed by a detailed, visual explanation of each parameter.

Parameter	Controls	Typical Use
Max tokens	Response length cap	Prevent runaway or cut-off generations
Temperature	Overall randomness	Low for factual tasks, high for creative tasks
Top-k	Candidate pool size (fixed count)	Limit sampling to a fixed number of top tokens
Top-p	Candidate pool size (probability mass)	Adaptive alternative to top-k
Frequency penalty	Repetition of already-frequent tokens	Reduce redundancy in summaries/reports
Presence penalty	Introduction of brand-new tokens	Encourage novelty in brainstorming
Stop sequences	Hard stop trigger	Enforce clean structured output boundaries

7 LLM Generation Parameters

Every generation from an LLM is shaped by parameters under the hood.

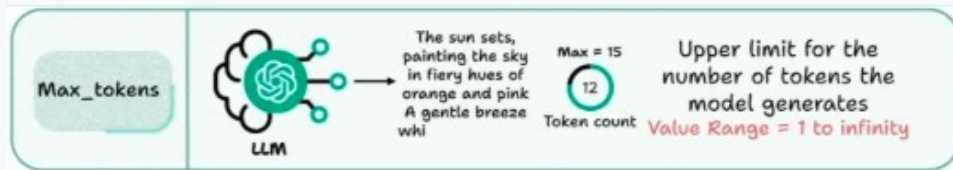
Knowing how to tune is important so that you can produce sharp and more controlled outputs.

Here are the 7 levers that matter most:

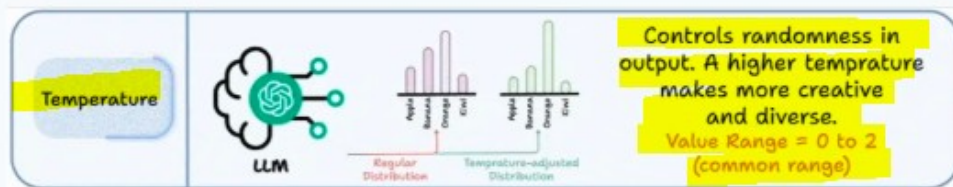
7 LLM Generation parameters

Max_tokens	 The sun sets, painting the sky in fiery hues of orange and pink. A gentle breeze whispers through the trees. Max = 15 Token count 15 Upper limit for the number of tokens the model generates Value Range = 1 to infinity
Temperature	 Controls randomness in output. A higher temperature makes more creative and diverse. Value Range = 0 to 2 (common range)
Top_p	 Controls probability distribution is considered when sampling tokens Value Range = 0 to 1
Top_k	 Limits the number of top probable tokens to sample from Value Range = 1 to infinity
Frequency penalty	 Dogs love to play and run and chase and bark and nap, and Penalizes token repetition based on frequency. Positive values reduce repetition Value Range = -2 to 2
Presence penalty	 Puppies nap. Kittens play. The sun shines. A bird sings. The day is perfect. Encourages the model to use new tokens that haven't been generated Value Range = -2 to 2
Stop	 There are some words restricted or forbidden, such as gambling. A list of tokens where the model will stop generating further tokens Value Range = Custom list

- **Max tokens**

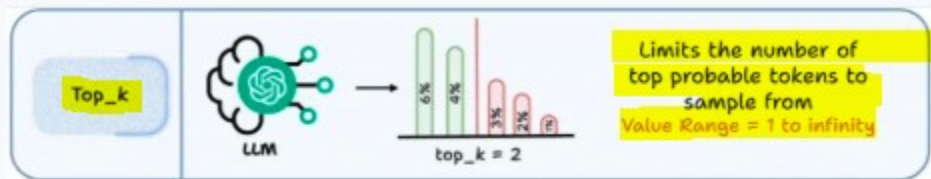


- This is a hard cap on how many tokens the model can generate in one response.
- Too low → truncated outputs; too high → could lead to wasted compute.
- **Temperature (covered in detail here):**



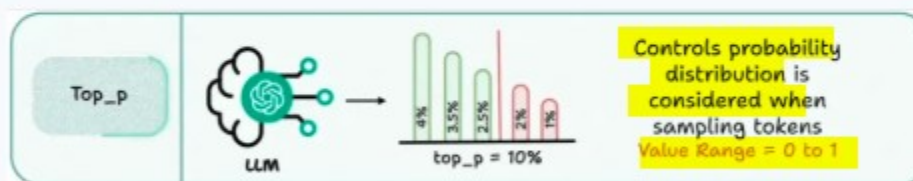
- Governs randomness. Low temperature (~0) makes the model deterministic.
- Higher temperature (0.7–1.0) boosts creativity, diversity, but also noise.
- Use case: lower for QA/chatbots, higher for brainstorming/creative tasks.

- **Top-k:**



- The default way to generate the next token is to sample from all tokens, proportional to their probability.
- This parameter restricts sampling to the top k most probable tokens.
- Example: $k=5 \rightarrow$ model only considers 5 most likely next tokens during sampling.
- Helps enforce focus, but overly small k may give repetitive outputs.

- **Top-p (nucleus sampling):**



- Instead of picking from all tokens or top k tokens, model samples from a probability mass up to p .
- Example: $\text{top}_p=0.9 \rightarrow$ only the smallest set of tokens covering 90% probability are considered.
- More adaptive than top_k , useful when balancing coherence with diversity.

- **Frequency penalty:**



- Reduces likelihood of reusing tokens that have already appeared frequently.
- Positive values discourage repetition, negative values exaggerate it.
- Useful for summarization (avoid redundancy) or poetry (intentional repetition).

- **Presence penalty**



- Encourages the model to bring in new tokens not yet seen in the text.
- Higher values push for novelty, lower values make the model stick to known patterns.
- Handy for exploratory generation where diversity of ideas is valued.

LLM Generation Parameters — Simple Interview Notes

Plain-English explanations for each parameter. No formulas — just what it does, why it matters, and a one-line way to say it in an interview.

0. The Big Picture (say this first if asked to explain generation)

"When an LLM generates text, at every single step it's not just picking the 'best' next word — it's looking at a ranked list of possible next words, each with a confidence score. These parameters control **how that list gets narrowed down** and **which word actually gets picked** from it. A few of them also control **when the model should stop talking**."

That's really it. Everything below is a variation on one of those two ideas: *narrowing the choices* or *deciding when to stop*.

1. Max Tokens

One-liner: "The word limit for the response."

Simple explanation: It's a hard cutoff on how long the response can be. Once the model hits that limit, it stops immediately — even if it was in the middle of a sentence.

Why it matters:

- Too low → answer gets cut off awkwardly.
- Too high → you might pay for/wait for more than you need.

Interview soundbite:

"Max tokens is just a length cap — it doesn't change what the model says, only how much of it you let it finish saying."

2. Temperature

One-liner: "How random or safe the model's word choices are."

Simple explanation: Think of the model as having a shortlist of possible next words, each more or less "confident." Temperature controls how much it's willing to gamble on a less-obvious choice.

- **Low temperature** → always picks the safest, most obvious word. Predictable, focused, sometimes boring.
- **High temperature** → more willing to pick surprising, creative words. More interesting, but riskier — can go off the rails.

Analogy:

"It's like asking someone a question — at low temperature they give you the textbook answer every time. At high temperature they start improvising and going off-script."

When to use what:

- Low temperature → coding, facts, customer support.
- High temperature → brainstorming, creative writing, poetry.

Interview soundbite:

"Temperature controls the model's willingness to take risks with word choice — low is predictable, high is creative but noisier."

3. Top-k

One-liner: "Only consider the top K most likely next words."

Simple explanation: Before picking a word, the model narrows its options down to a fixed number — say, the top 10 most likely candidates — and only chooses from that shortlist. Everything else is thrown away, no matter how the rest of the list looks.

Analogy:

"It's like a hiring manager who always looks at exactly the top 5 resumes, no matter how many great candidates there really are or aren't."

Trade-off:

- Too small K → responses feel repetitive and robotic.
- Reasonable K → keeps things focused without being too rigid.

Interview soundbite:

"Top-k limits the model to picking from a fixed number of top candidates — it's a simple, blunt way to cut out unlikely words."

4. Top-p (Nucleus Sampling)

One-liner: "Only consider enough top words to cover, say, 90% of the confidence."

Simple explanation: Instead of a fixed number of words like top-k, top-p keeps adding the next most-likely words until their combined confidence adds up to a target percentage (e.g., 90%). Then it picks from just that group.

Why it's smarter than top-k: It automatically adjusts. If the model is very sure about the next word, the shortlist might just be 1–2 words. If the model is unsure, the shortlist naturally grows to include more options. Top-k can't do that — it always uses the same fixed count.

Analogy:

"Instead of always looking at exactly 5 resumes, the hiring manager keeps reading resumes until they've covered '90% of the good candidates' — sometimes that's 2 resumes, sometimes it's 15, depending on how competitive the pool is."

Interview soundbite:

"Top-p is the adaptive version of top-k — instead of a fixed count of words, it uses a fixed amount of confidence."

5. Frequency Penalty

One-liner: "Discourages the model from repeating the same words too much."

Simple explanation: The more a word has already been used in the response, the more the model gets discouraged from using it again. The penalty builds up the more times a word repeats.

Analogy:

"Like a teacher telling a student, 'You've already used the word "amazing" four times — try something else.' The more you overuse a word, the stronger the nudge to stop."

When to use it:

- Turn it up for summaries/reports where you don't want redundant phrasing.
- Turn it down (or make it negative) if some repetition is intentional, like in poetry.

Interview soundbite:

"Frequency penalty punishes words based on how *often* they've already shown up — the more repeats, the bigger the penalty."

6. Presence Penalty

One-liner: "Encourages the model to bring in new words/topics it hasn't used yet."

Simple explanation: This one doesn't care *how many times* a word has been repeated — it only checks *has this word shown up at all yet?* If yes, it gets a small, flat penalty, encouraging the model to explore something new instead.

How it's different from frequency penalty:

- Frequency penalty = "you've used this word 5 times, stop."

- Presence penalty = "you've used this word even once — try mixing in something new."

Analogy:

"It's like a brainstorming coach saying 'you already mentioned that idea — what's something you haven't brought up yet?' — it doesn't matter if you mentioned it once or five times, the nudge is the same."

When to use it: Great for brainstorming or idea generation, where you want breadth of topics rather than depth on one thing.

Interview soundbite:

"Presence penalty nudges the model toward new topics it hasn't touched yet, regardless of how many times it repeated something else."

7. Stop Sequences

One-liner: "Specific words/symbols that tell the model 'stop right here.'"

Simple explanation: You give the model a list of trigger strings (like "\n\n", "END", or "}"). The moment the model produces one of those, generation halts immediately — no matter what else it might have been about to say.

Why it matters: Super useful when you need clean, structured output — like JSON — where you don't want the model to add extra commentary after the actual answer is done.

Analogy:

"It's like giving someone a magic word — the second they say it, they have to stop talking, mid-thought if necessary."

Interview soundbite:

"Stop sequences are a hard, rule-based stop trigger — unlike max tokens, which is just a length cap, this stops generation based on *content*, not count."

Quick Comparison Table (great for a whiteboard answer)

Parameter	What it controls	Simple mental model
Max tokens	Response length	"Word limit"
Temperature	Randomness/creativity	"How much it gambles on unusual words"

Parameter	What it controls	Simple mental model
Top-k	Candidate pool (fixed size)	"Only look at the top K options"
Top-p	Candidate pool (adaptive size)	"Only look at options covering X% confidence"
Frequency penalty	Repetition (scales with count)	"Stop repeating that word so much"
Presence penalty	Topic diversity (flat, one-time)	"Bring in something new"
Stop sequences	Hard stop trigger	"Stop the moment you see this"

Likely Interview Questions + Simple Answers

Q: What's the difference between top-k and top-p?

"Top-k always looks at a fixed number of word choices. Top-p instead looks at however many words it takes to cover a target confidence level — so it adjusts automatically depending on how sure the model is."

Q: What's the difference between frequency penalty and presence penalty?

"Frequency penalty cares about *how many times* something's been repeated — the more repeats, the bigger the discouragement. Presence penalty is simpler — it just checks *has this been said before, yes or no* — and nudges toward new material either way."

Q: When would you use a low vs. high temperature?

"Low temperature for anything that needs to be accurate and consistent — like code or factual Q&A. High temperature for anything creative — like brainstorming or storytelling — where you want variety over precision."

Q: Why would you use stop sequences instead of just max tokens?

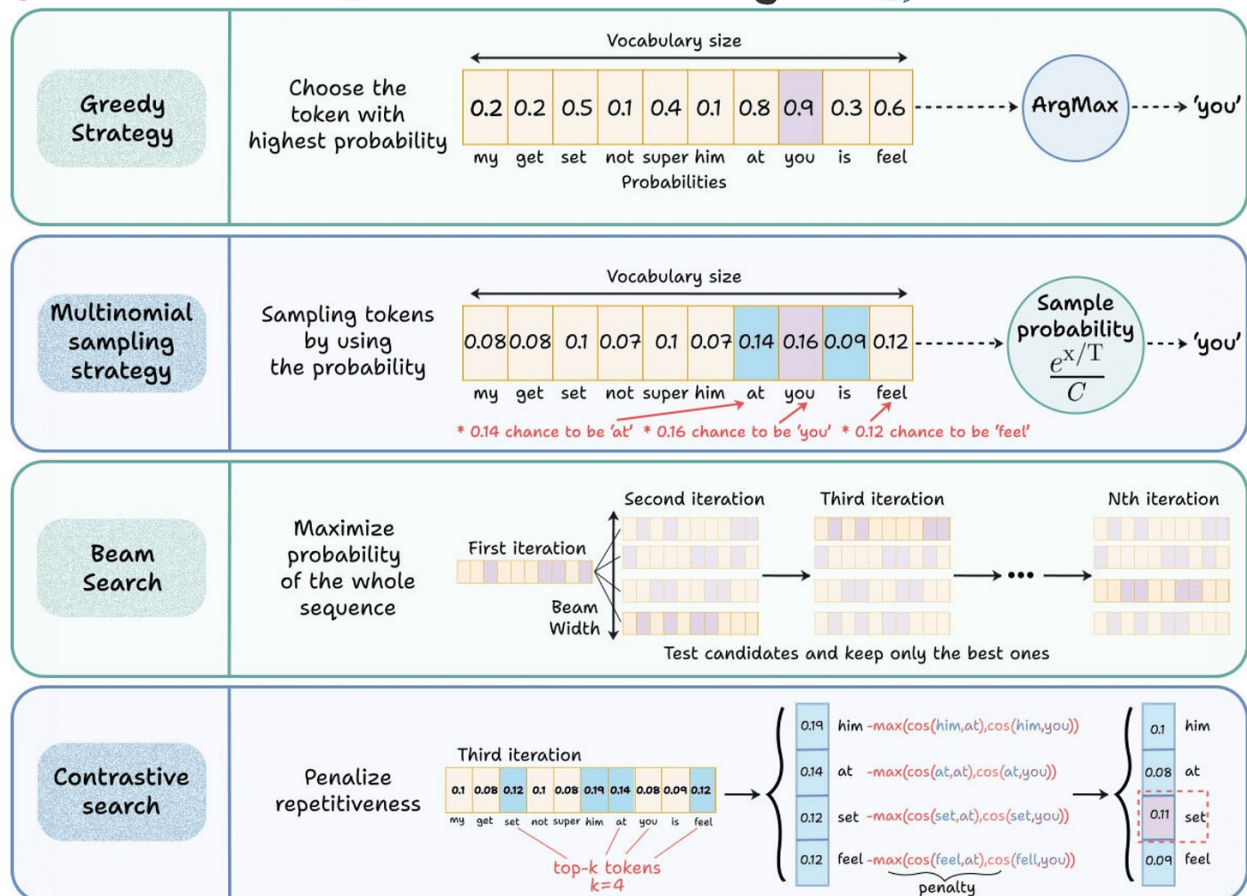
"Max tokens just cuts things off after a certain length, which can chop a sentence in half. Stop sequences let you end generation cleanly at a meaningful point — like right after a closing bracket in JSON — so the output stays clean and structured."

Q: Can you combine these parameters?

"Yes, they're not exclusive — a typical setup might use a moderate temperature together with top-p to balance creativity and coherence, plus stop sequences to keep the output clean."

4 LLM Text Generation Strategies

4 LLM Text Generation Strategies mcp.DailyDoseofDS.com



"When an LLM generates text, it predicts one word at a time, not the whole sentence at once. But having probabilities for the next word isn't enough — we need a strategy to actually decide which word to pick. I like to think of this like **planning a road trip, one town at a time**, where at each town, you have several roads to choose from, each with a different 'promise' score."

1. Greedy Search

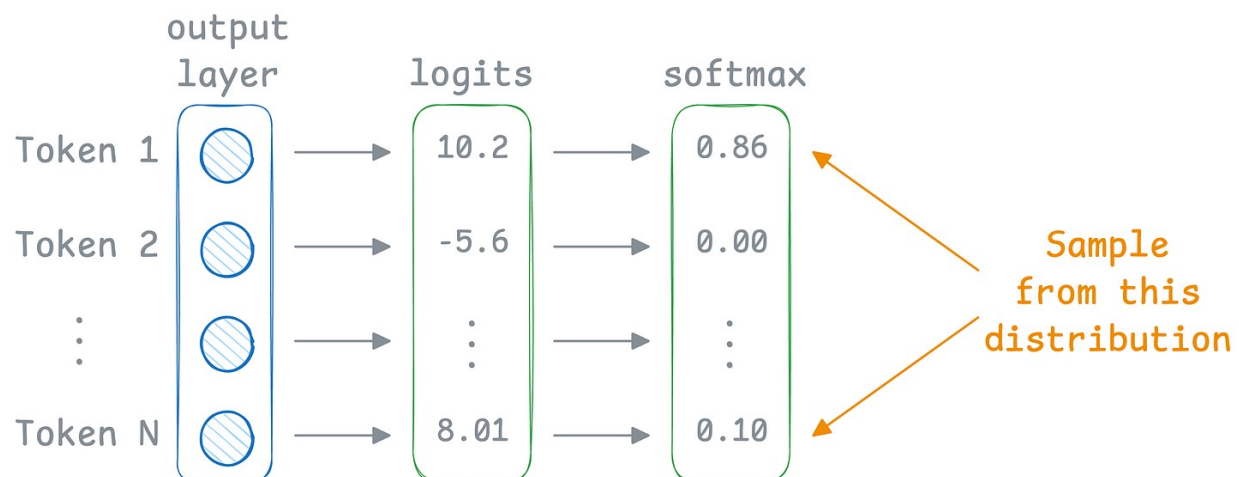
The naive approach greedily chooses the word with the highest probability from the probability vector, and autoregresses. This is often not ideal since it leads to repetitive sentences.

Explanation: Always picks the single **highest-probability** word at each step. Fast and simple, but leads to repetitive, dull output.

Analogy: "It's like always taking the road that *looks* best right this second — the shiny, wide, freshly-paved one — without thinking ahead. You keep taking the 'obvious' road at every town, and you end up going in loops or hitting dead ends, because you never explored alternatives."

2. Multinomial Sampling

Instead of always picking the top token, we can sample from the probability distribution available in the probability vector.



Explanation: Instead of always taking the top word, **we randomly sample based on the probability distribution** — better words have a higher chance but it's not guaranteed. Controlled by **temperature**: higher = more random/creative, lower = more predictable.

Analogy: "This is like spinning a weighted wheel at each town — good roads get bigger slices of the wheel, so you'll usually pick a decent road, but sometimes you'll take a surprising detour. Turn up the 'temperature' and the wheel becomes more evenly spread out (more adventurous trips); turn it down and you almost always pick the safe road."

3. Beam Search

Both approach 1 and approach 2 have a problem. They only focus on the most immediate token to be generated. Ideally, we care about maximizing the probability of the whole sequence, not just the next token.

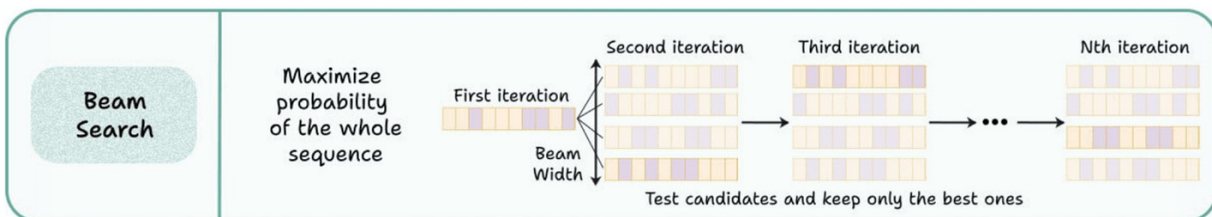
To maximize this product, you'd need to know future conditionals (what comes after each candidate).

But when decoding, we only know probabilities for the next step, not the downstream continuation.

Beam search tries to approximate the true global maximization:

- At each step, it expands the top k partial sequences (the beam).
- Some beams may have started with less probable tokens initially, but lead to much higher-probability completions.
- By keeping alternatives alive, beam search explores more of the probability tree.

This is widely used in tasks like machine translation, where correctness matters more than creativity.



Explanation: Greedy and sampling only look one step ahead. Beam search keeps multiple candidate paths alive at once (say, top 3-5), expands all of them, and at the end picks whichever full path has the best overall score. Great for tasks needing accuracy, like translation.

Analogy: "Instead of committing to one road immediately, you send out 3-4 scout cars down different roads at the same time. Some scouts might start on a road that doesn't look amazing but eventually leads to the best destination. At the end of the trip, you compare all the scouts' routes and pick whichever one had the best overall journey — not just the best first turn."

4. Contrastive Search

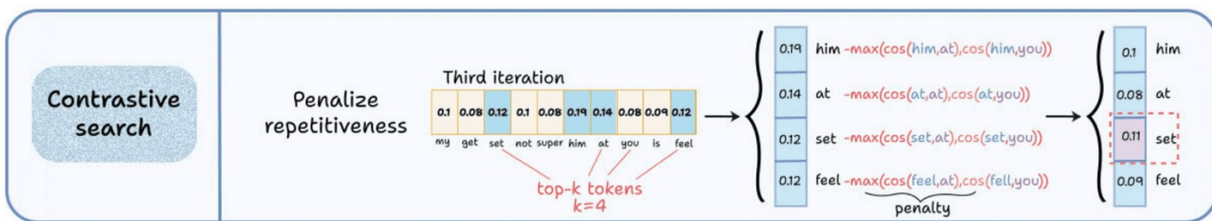
This is a newer method that balances fluency with diversity.

Essentially, it penalizes repetitive continuations by checking how similar a candidate token is to what's already been generated to have more diversity in the output.

- At each step, the model considers candidate tokens.
- Applies a penalty if the token is too similar to what's already been generated.
- Selects the token that balances probability and diversity.

This way, it also prevents "stuck in a loop" problems while keeping coherence high.

It's especially effective for longer generations like stories, where repetition can easily creep in.



Explanation: A newer method balancing fluency and diversity. It penalizes words too similar to what's already been generated, preventing repetition — especially useful in long text like stories.

Analogy: "This is like driving with a rule: 'don't take a road that looks too much like one I already drove.' You still prefer good roads, but you actively avoid backtracking or circling the same neighborhood — keeping the trip fresh and moving forward instead of getting stuck looping through the same town."

Closing summary (with analogy)

"So overall — greedy takes the closest exit even if it's a dead end, sampling adds a bit of adventurous randomness, beam search sends out scouts to find the best overall route, and contrastive search avoids retracing your steps. The right strategy depends on the destination: beam search when you need the *correct* route (like translation), and sampling or contrastive search when you want a *creative*, non-repetitive journey (like storytelling)."

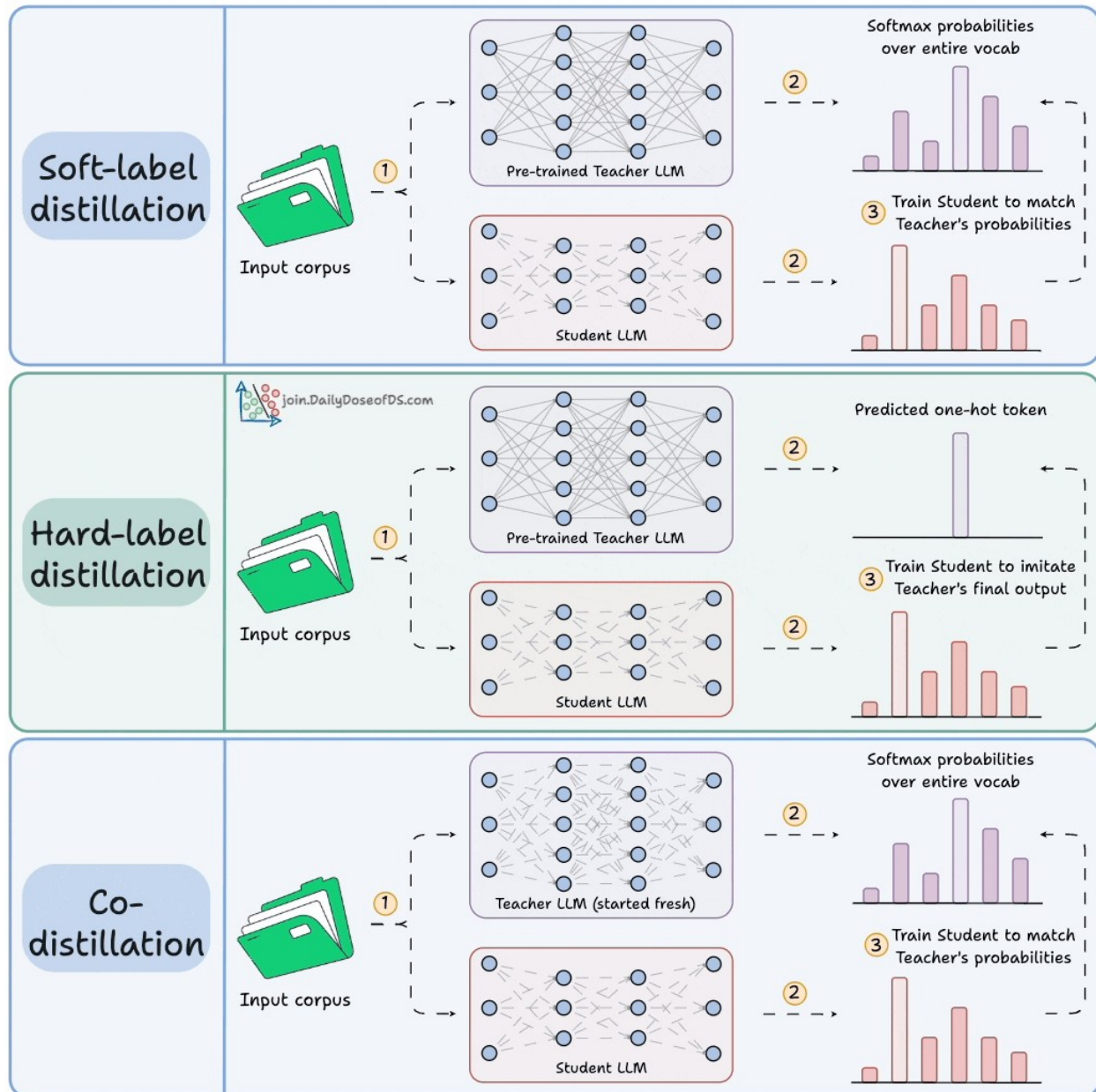
3 Techniques to Train An LLM Using Another LLM

LLMs don't just learn from raw text; they also learn from each other:

- Llama 4 Scout and Maverick were trained using Llama 4 Behemoth.
- Gemma 2 and 3 were trained using Google's proprietary Gemini.

Distillation helps us do so, and the visual below depicts three popular techniques.

3 Techniques to Train an LLM using Another LLM



The idea is to transfer "knowledge" from one LLM to another, which has been quite common in traditional deep learning ([like we discussed here](#)).

Distillation in LLMs can happen at two stages:

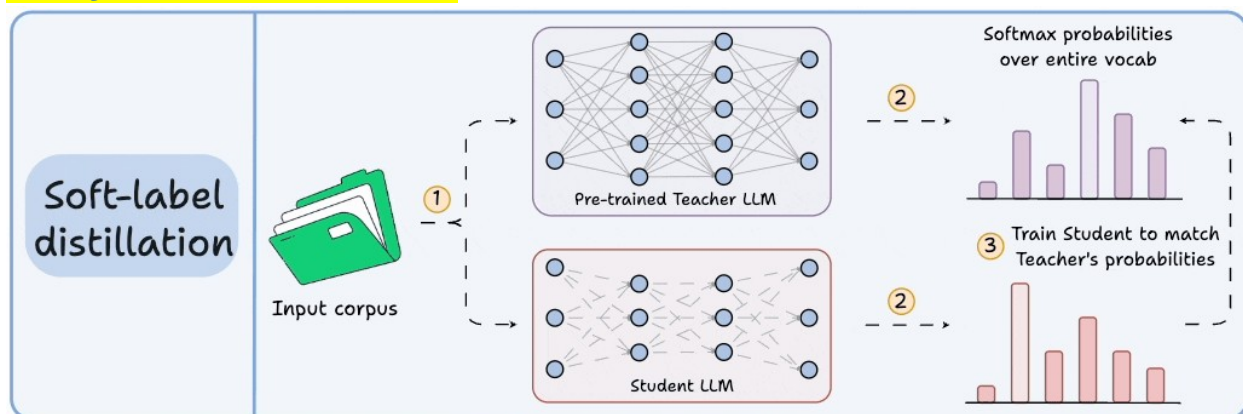
1) Pre-training

- Train the bigger Teacher LLM and the smaller student LLM together.
- Llama 4 did this.

2) Post-training:

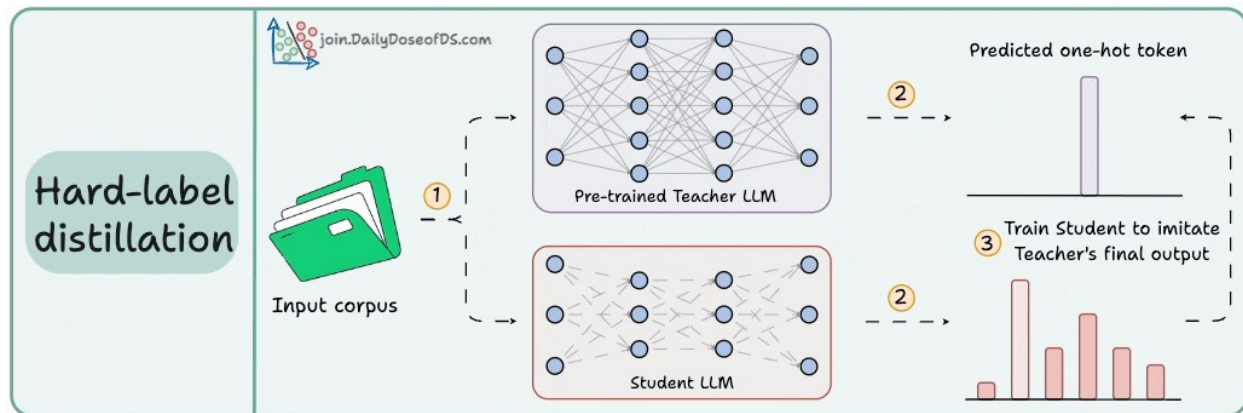
- Train the bigger Teacher LLM first and distill its knowledge to the smaller student LLM.
 - DeepSeek did this by distilling DeepSeek-R1 into Qwen and Llama 3.1 models.
- You can also apply distillation during both stages, which Gemma 3 did.
Here are the three commonly used distillation techniques:

1) Soft-label distillation:



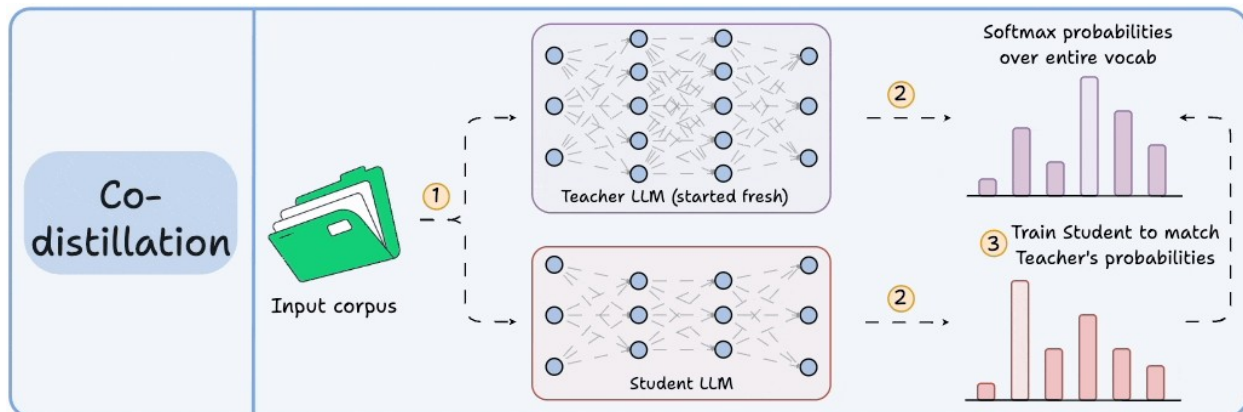
- Use a fixed pre-trained Teacher LLM to generate softmax probabilities over the entire corpus.
- Pass this data through the untrained Student LLM as well to get its softmax probabilities.
- Train the Student LLM to match the Teacher's probabilities.
Visibility over the Teacher's probabilities ensures maximum knowledge (or reasoning) transfer. However, you must have access to the Teacher's weights to get the output probability distribution.
Even if you have access, there's another problem!
Say your vocab size is 100k tokens and your data corpus is 5 trillion tokens.
Since we generate softmax probabilities of each input token over the entire vocabulary, you would need 500 million GBs of memory to store soft labels under float8 precision.
The second technique solves this.

2) Hard-label distillation



- Use a fixed pre-trained Teacher LLM to just get the final one-hot output token.
- Use the untrained Student LLM to get the softmax probabilities from the same data.
- Train the Student LLM to match the Teacher's probabilities.
DeepSeek did this by distilling DeepSeek-R1 into Qwen and Llama 3.1 models.

3) Co-distillation



- Start with an untrained Teacher LLM and an untrained Student LLM.
- Generate softmax probabilities over the current batch from both models.
- Train the Teacher LLM as usual on the hard labels.
- Train the Student LLM to match its softmax probabilities to those of the Teacher.
Llama 4 did this to train Llama 4 Scout and Maverick from Llama 4 Behemoth.
Of course, during the initial stages, soft labels of the Teacher LLM won't be accurate.
That is why Student LLM is trained using both soft labels + ground-truth hard labels.

:

"Distillation is basically transferring knowledge from a bigger, smarter model (Teacher) to a smaller model (Student), so the Student can perform almost as well without needing the same size or compute. It's like a student learning from an expert teacher instead of learning everything from scratch by reading raw textbooks."

"So basically, the idea here is — you don't always need to train an LLM from scratch using raw text. You can also make one LLM learn from another LLM that's already trained. This is called **distillation** — think of it like a smaller model 'copying' knowledge from a bigger, smarter model.

A few real examples — Llama 4's smaller models, Scout and Maverick, were trained using a much bigger model called Behemoth. Google's Gemma models were trained using Gemini. And DeepSeek did something similar too.

Now, there are basically **3 ways** people do this distillation."

1. *Soft-label distillation*

"In this one, the teacher model doesn't just say 'this is the answer' — it shows its **full confidence spread**. Like, 'I'm 70% sure it's this word, 20% sure it's that word, 10% something else.' And the student model tries to copy that entire confidence pattern, not just the final answer.

It's great because the student learns a lot more nuance — almost like learning *how* the teacher thinks, not just *what* it says.

But the problem is — storing that much information for every single word, across a huge dataset, is insanely expensive. Like, we're talking hundreds of millions of GBs kind of expensive. So it doesn't scale well."

2. *Hard-label distillation*

"This one's simpler — the teacher just gives the **final answer**, no confidence breakdown, nothing. Just 'the answer is this.' The student trains to match that.

It's way lighter on memory, way easier to scale. And this is actually what DeepSeek did — they took their big R1 model and distilled it down into smaller models like Qwen and Llama 3.1."

3. Co-distillation

"This one's a bit different — here, **both** the teacher and student start off untrained. They're basically learning together, side by side. The teacher trains normally on the real correct answers, and the student tries to copy the teacher's outputs as it goes.

But since early on the teacher itself isn't great yet, the student also keeps checking against the actual ground-truth answers too — so it's not blindly trusting the teacher.

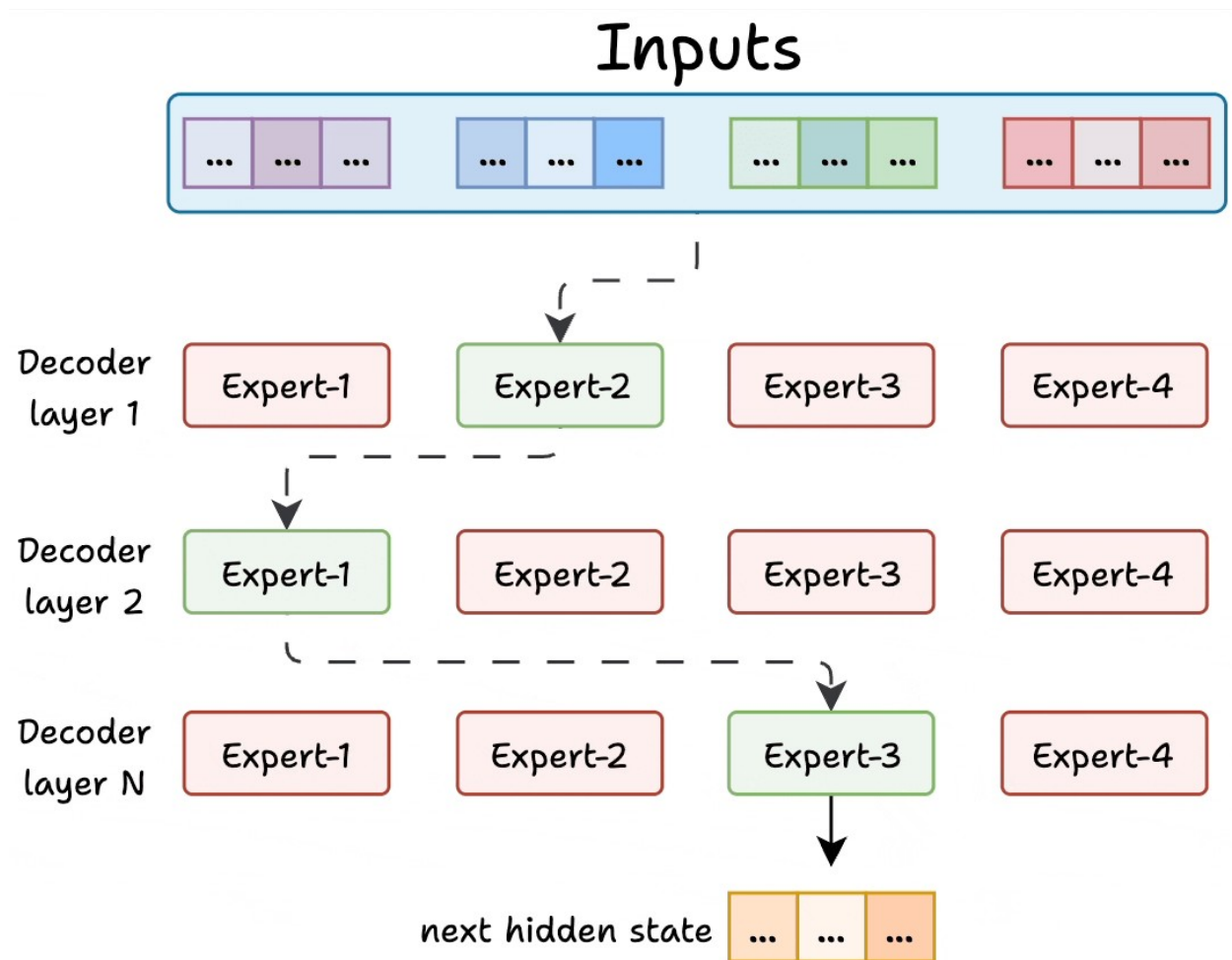
This is what Llama 4 did to train Scout and Maverick from Behemoth."

Quick way to wrap it up

"So basically — soft-label gives the richest info but is super heavy on memory, hard-label is lightweight and just transfers the final answer, and co-distillation trains both models together in real time. Which one you use really depends on how much compute and memory you have, and how much depth of knowledge you want to transfer."

Transformer vs. Mixture of Experts (MoE) in LLMs

- Transformer uses a feed-forward network.
- MoE uses experts, which are feed-forward networks but smaller compared to that in Transformer.
During inference, a subset of experts are selected. This makes inference faster in MoE. Also, since the network has multiple decoder layers:



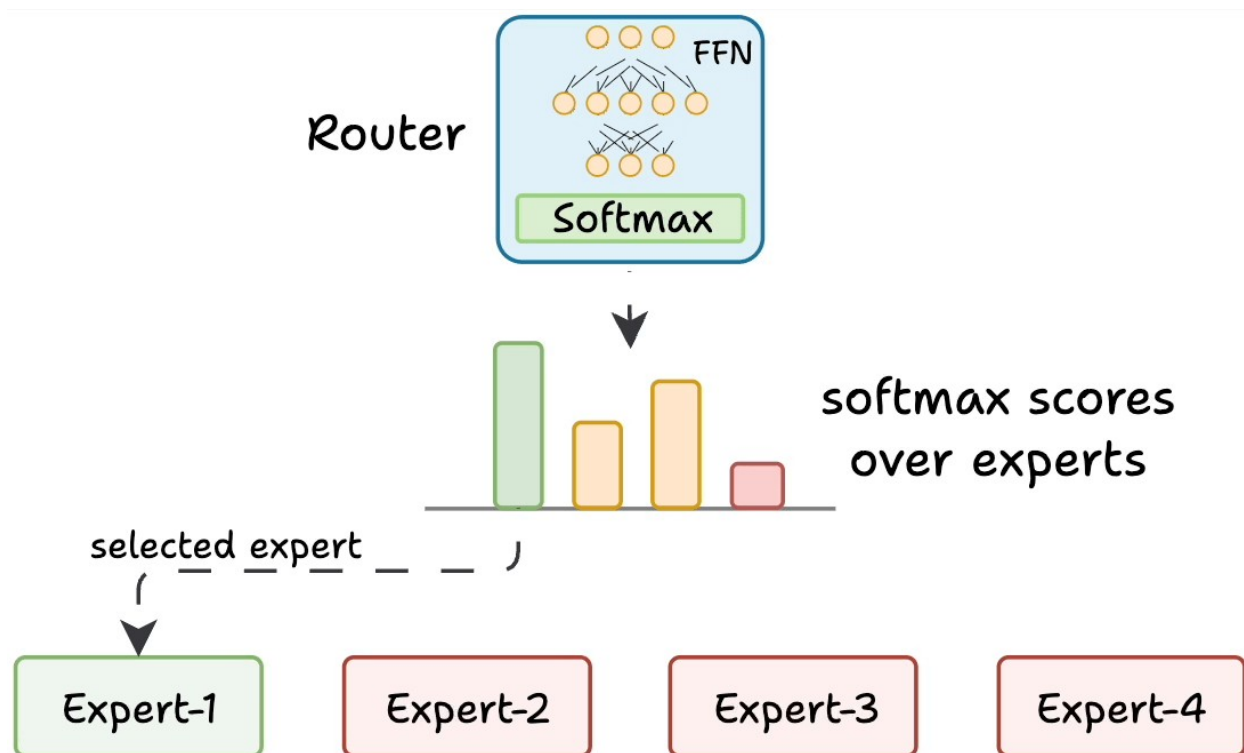
- the text passes through different experts across layers.
- the chosen experts also differ between tokens.

But how does the model decide which experts should be ideal?

The router does that.

The router is like a multi-class classifier that produces softmax scores over experts. Based on the scores, we select the top **K** experts.

The router is trained with the network and it learns to select the best experts.



Core Idea

The main difference between a regular Transformer and an MoE model is in the **decoder block**:

- **Transformer** → uses one feed-forward network (FFN) per layer.
- **MoE** → replaces that single FFN with multiple smaller FFNs called "**experts**." During inference, only a subset of these experts is actually used.

This selective activation makes MoE models **faster at inference**, even though they have more total parameters.

How Experts Get Selected — The Router

- A component called the **router** decides which experts to use.
- The router acts like a multi-class classifier — it produces softmax scores over all experts.
- Based on these scores, the **top K experts** are selected for each token.
- Since there are multiple decoder layers, tokens can pass through **different experts at each layer**, and different tokens can pick different experts.
- The router is trained along with the rest of the network.

Challenge 1: Expert Imbalance (Rich-Get-Richer Problem)

Early in training, all experts look similar, so one gets picked somewhat randomly (say Expert 2). Because it gets used, it gets a bit better → gets picked again → gets even better → keeps getting picked. Meanwhile, other experts remain under-trained.

Fix (2-part solution):

1. Add **noise** to the router's output so other experts get a fair chance at higher scores.
2. Set all logits **except the top K** to -infinity — after softmax, these become zero, forcing variety in selection over time.

Challenge 2: Uneven Token Load

Some experts may end up processing far more tokens than others, causing imbalance again.

Fix: Set a **limit on how many tokens** an expert can process. Once an expert hits its limit, extra tokens are routed to the **next-best expert** instead.

Key Outcome

- MoE models have **more total parameters** than a standard Transformer, but only a **fraction are active** at any time (since only top K experts are used).
 - This leads to **faster inference** despite the larger overall model size.
 - **Example real-world model:** Mixtral 8x7B by Mistral AI.
-

Simple Explanation with Analogy (Interview-Ready)

Analogy: A Hospital With Specialist Doctors

"Think of a **Transformer** like a hospital with just **one general physician** — every patient (token), no matter what problem they have, goes to this same doctor. It works, but the doctor has to know a bit of everything, and it's not maximally efficient.

Now, **MoE** is like a hospital with a team of **specialist doctors** — a cardiologist, a dermatologist, an orthopedic, etc. When a patient comes in, a **receptionist** (this is the router) looks at their symptoms and directs them to the 2-3 most relevant specialists, instead of sending them to every doctor in the building.

So instead of one generalist handling everyone, you've got many specialists, but each patient only sees a couple of them. That's why it's faster — you're not activating the whole hospital for every patient, just the relevant experts."

How I'd Explain It Out Loud in an Interview

"So the key difference between a regular Transformer and a Mixture of Experts model is in the feed-forward part of the decoder. A normal Transformer has one feed-forward network that every token passes through. MoE replaces that with multiple smaller feed-forward networks called 'experts' — and for each token, only a few of these experts actually get used.

There's a **router** — think of it like a receptionist — that looks at each token and decides which experts should handle it. It gives a score to every expert, kind of like a classification problem, and then picks the top K experts based on those scores.

Now, this creates a couple of real challenges during training.

First problem — early on, since all experts start off pretty similar, one expert might randomly get picked a bit more. Because it gets more practice, it becomes slightly better, so it keeps getting picked again and again — kind of like the 'rich getting richer.' Meanwhile, other experts barely get used and stay undertrained.

To fix this, two things are done — you add some **random noise** to the router's scoring so other experts also get a fair shot at being picked, and you also force only the top K experts to be usable at all — everything else gets zeroed out. This keeps forcing variety over time.

Second problem — even if experts are picked more evenly, some might still end up handling way more tokens overall than others. So there's usually a **cap** — like, 'this expert can only handle this many tokens' — and once it hits that limit, extra tokens get redirected to the next best expert instead.

End result — yeah, MoE models technically have way more parameters overall, but since only a few experts activate per token, inference ends up being **faster** compared to a dense Transformer of similar size. Mixtral 8x7B is a well-known model built this way."

Quick Closing Line for Interviews

"So basically — Transformer is like one generalist doctor seeing every patient, while MoE is like a hospital with a team of specialists, where a router acts like a receptionist directing each patient to just the right couple of specialists. This makes MoE more efficient at inference, even though it holds way more total knowledge/parameters overall."